Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · WEEK 5

CPU Scheduling Algorithms

Which process runs next, how to judge a scheduling algorithm, and the algorithms real systems use.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Describe the common CPU scheduling algorithms.
2. Assess an algorithm against the scheduling criteria.
3. Explain the issues in multiprocessor and multicore scheduling.
4. Describe real-time scheduling algorithms.
5. Describe the schedulers of Linux, Windows and Solaris, and evaluate algorithms by modelling and simulation.

THIS WEEK

Topic 5

Many processes are ready and there are few CPU cores. Every few milliseconds the operating system must choose.

DISCUSSION

Who should go first at a shared printer?

The first to arrive? The shortest job? The most important person? Each answer is a scheduling algorithm, and each has a cost.

PART 1

Basic concepts

Processes alternate between using the CPU and waiting for I/O.

The CPU–I/O burst cycle

ONE PROCESS OVER TIME (IDLE = WAITING FOR I/O)



The cycle

Execution is a cycle of CPU bursts and I/O waits. It starts and ends with a CPU burst.

Many short bursts

Measured burst times show a large number of short CPU bursts and a small number of long ones.

Two kinds of program

An I/O-bound program has many short CPU bursts. A CPU-bound program has a few long ones.

The CPU scheduler

It selects one process from the *ready queue* and allocates a CPU core to it.

	A scheduling decision may happen when a process…	Kind
1	switches from running to waiting, for I/O for example	Nonpreemptive
2	switches from running to ready, after an interrupt	Preemptive
3	switches from waiting to ready, when its I/O completes	Preemptive
4	terminates	Nonpreemptive

Preemptive and nonpreemptive scheduling

KEEP UNTIL DONE

Nonpreemptive

Once a process has the CPU, it keeps it until it terminates or switches to waiting. Simple, but one long process can hold everyone up.

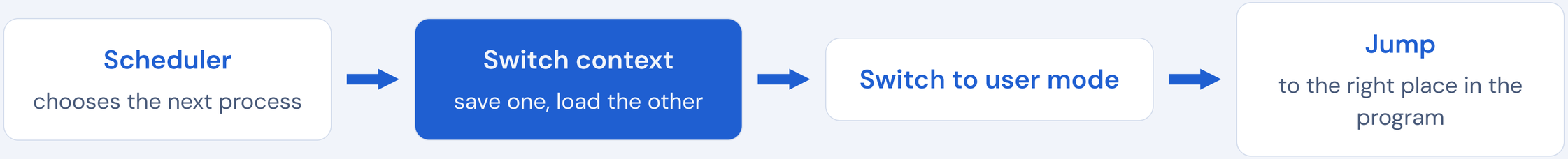
CAN BE INTERRUPTED

Preemptive

The operating system can take the CPU away. Windows, macOS, Linux and UNIX all schedule this way. It can cause race conditions when processes share data.

A process preempted in the middle of updating shared data leaves that data inconsistent for the next process. Week 6 is about that problem.

The dispatcher



The dispatcher gives control of the CPU to the chosen process. Dispatch latency is the time it takes to stop one process and start another. It should be as short as possible.

Scheduling criteria

Criterion	What it measures	We want
CPU utilization	How busy the CPU is kept	Maximum
Throughput	Processes completed per unit of time	Maximum
Turnaround time	Time from submission of a process to its completion	Minimum
Waiting time	Total time a process spends in the ready queue	Minimum
Response time	Time from a request to the first response, not the full output	Minimum

In the examples that follow we compare algorithms by their average waiting time.

PART 2

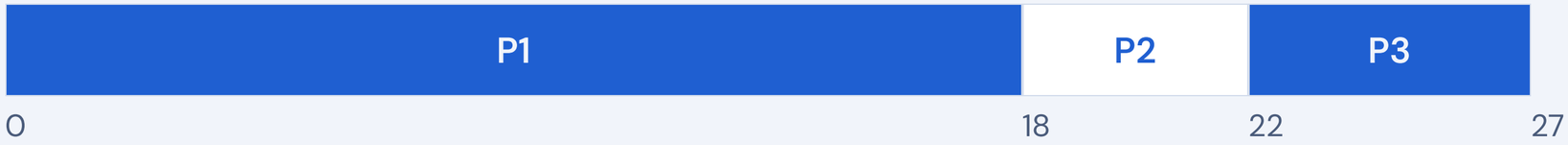
Scheduling algorithms

Six ways to choose from the ready queue.

First-come, first-served

Process	Burst time
P1	18
P2	4
P3	5

GANTT CHART



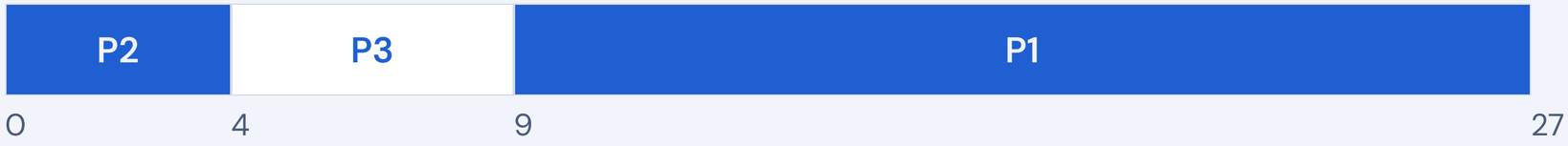
Waiting times: P1 = 0, P2 = 18, P3 = 22. Average = 40 / 3 = 13.3.

The process that asks for the CPU first gets it first. A FIFO queue is all it needs. It is nonpreemptive.

The same processes in another order

Process	Burst time
P2	4
P3	5
P1	18

GANTT CHART



Waiting times: P2 = 0, P3 = 4, P1 = 9. Average = 13 / 3 = 4.3.

The convoy effect: short processes stuck behind one long process. With FCFS the average depends heavily on the arrival order.

Shortest-job-first

Give the CPU to the process with the *shortest next CPU burst*.

Optimal

For a given set of processes, SJF gives the minimum average waiting time.

The difficulty

We cannot know the length of the next CPU burst. We can only estimate it, or ask the user.

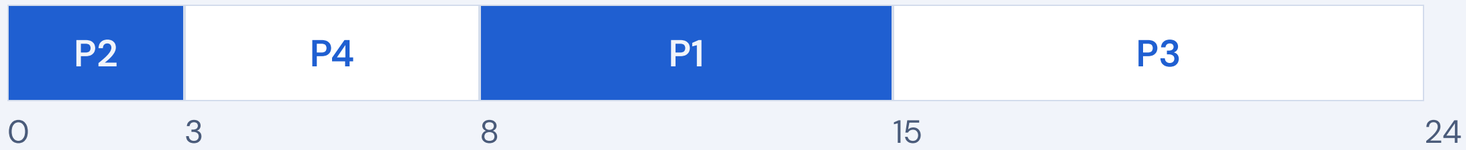
Two versions

Nonpreemptive, or preemptive. The preemptive version is called shortest-remaining-time-first.

SJF: an example

Process	Burst time
P1	7
P2	3
P3	9
P4	5

GANTT CHART



Waiting times: P2 = 0, P4 = 3, P1 = 8, P3 = 15. Average = 26 / 4 = 6.5.

All four arrive at time 0. In FCFS order the waiting times would be 0, 7, 10 and 19, an average of 9.

Estimating the next CPU burst

$$\tau(n+1) = \alpha \cdot t(n) + (1 - \alpha) \cdot \tau(n)$$

t(n) is the length of the burst just measured. τ(n) was our estimate for it. α, between 0 and 1, sets how much weight the recent past gets.

With α = 0 recent history is ignored. With α = 1 only the last burst counts. A common choice is α = 1/2.

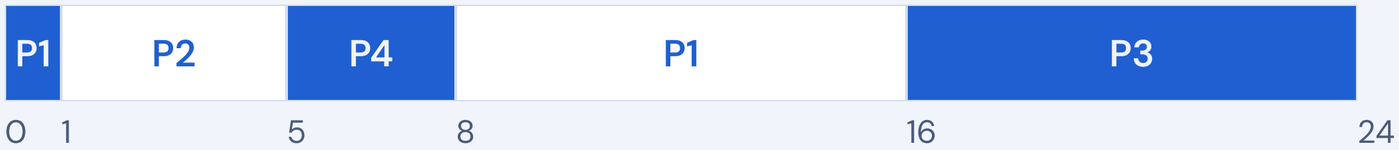
n	Measured t(n)	Estimate τ(n)
0	6	10
1	4	8
2	6	6
3	…	6

This is exponential averaging. Each older burst counts for less than the one after it. Here α = 1/2 and the first estimate is 10.

Shortest-remaining-time-first

Process	Arrival	Burst
P1	0	9
P2	1	4
P3	2	8
P4	3	3

GANTT CHART



Waiting times: P1 = 7, P2 = 0, P3 = 14, P4 = 2. Average = 23 / 4 = 5.75.

Preemptive SJF. When P2 arrives at time 1 with a burst of 4, P1 still needs 8, so P2 takes the CPU.

Round robin

Each process gets a small unit of CPU time, a *time quantum*, then goes to the back of the queue.

Fair by construction

With n processes and a quantum q, each gets 1/n of the CPU and no process waits more than (n - 1) × q.

A large quantum

Round robin behaves like first-come, first-served.

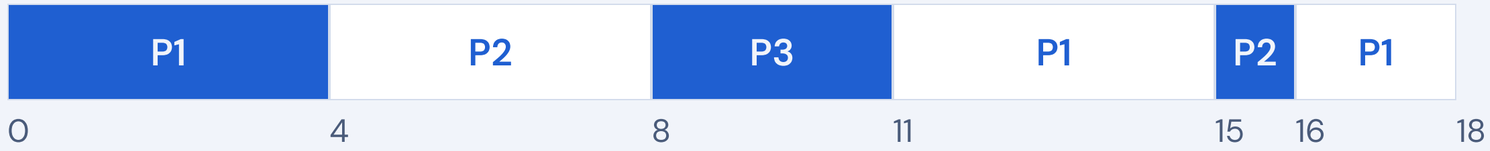
A small quantum

Too many context switches. The quantum must be large compared with the time of a switch.

Round robin: an example with a quantum of 4

Process	Burst time
P1	10
P2	5
P3	3

GANTT CHART



Waiting times: P1 = 8, P2 = 11, P3 = 8. Average = 27 / 3 = 9.

Round robin usually has a higher average turnaround than SJF, but a better response time. That is why interactive systems use it.

Choosing the time quantum

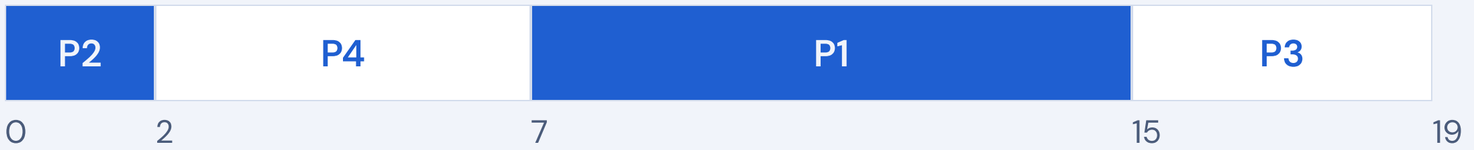
Quantum	Context switches for one burst of 10	Comment
12	0	The burst finishes inside one quantum
6	1	
1	9	Most of the time may go to switching

A rule of thumb: 80 percent of the CPU bursts should be shorter than the quantum. A quantum is usually 10 to 100 milliseconds; a context switch takes less than 10 microseconds.

Priority scheduling

Process	Burst	Priority
P1	8	3
P2	2	1
P3	4	4
P4	5	2

GANTT CHART



Waiting times: P2 = 0, P4 = 2, P1 = 7, P3 = 15. Average = 24 / 4 = 6.

The CPU goes to the process with the highest priority. Here the smallest number is the highest priority. SJF is priority scheduling where the priority is the predicted next burst.

Starvation and aging



Starvation

A low-priority process may never run, because higher-priority processes keep arriving.



Aging

The solution: as time passes, increase the priority of a process that has been waiting.



Priority with round robin

Run the highest-priority process. Processes with the same priority share the CPU by round robin.

Multilevel queue

Real-time processes
highest priority

System processes

Interactive processes

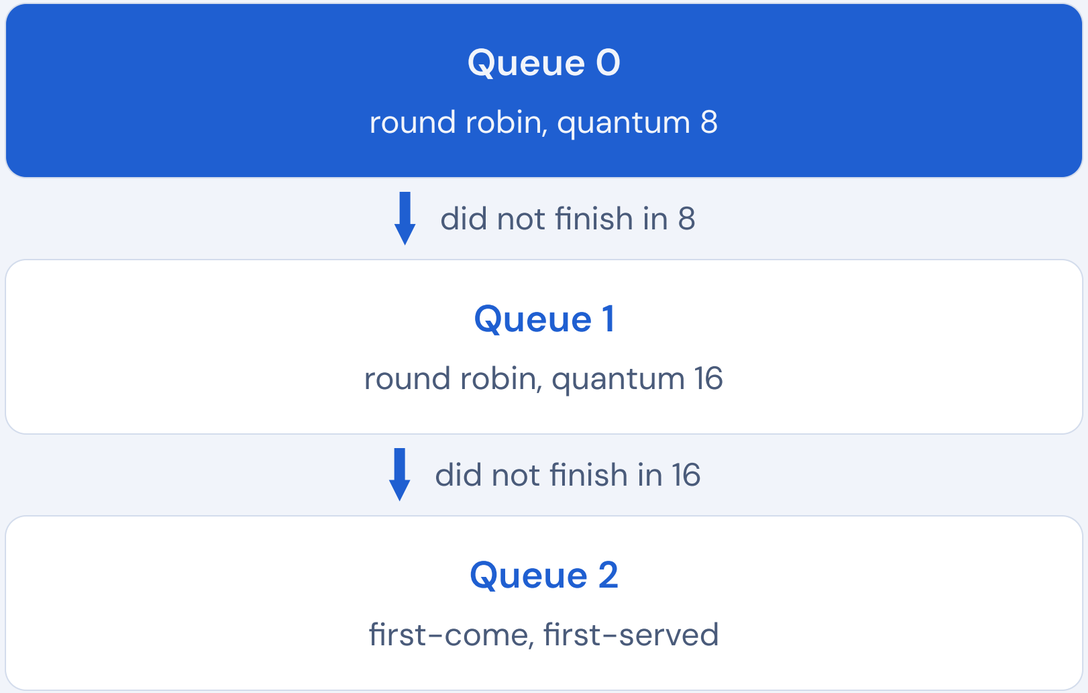
Batch processes
lowest priority

The ready queue is split into several queues, one for each kind of process.

A process stays in its queue permanently. Each queue has its own algorithm: round robin for interactive work, FCFS for batch.

Between the queues: fixed priority, where a lower queue runs only when the higher ones are empty, or a share of CPU time for each queue.

Multilevel feedback queue



A process can move between queues.

A new process enters queue 0. If it uses its whole quantum, it is moved down. Short, interactive processes finish near the top. Long, CPU-bound ones sink.

Aging can move a process that has waited too long back up.

The most general scheme, and the most complex: the number of queues, the algorithm of each, and the rules for moving up and down must all be chosen.

PART 3

Thread and multiprocessor scheduling

On a modern system it is kernel threads that are scheduled, on several cores.

Thread scheduling

PCS

Process-contention scope

The thread library schedules user threads onto the available lightweight processes. The competition is among the threads of one process.

SCS

System-contention scope

The kernel schedules kernel threads onto the CPUs. The competition is among all the threads in the system. Linux and macOS allow only this.

In Pthreads the scope is chosen when a thread is created: PTHREAD_SCOPE_PROCESS or PTHREAD_SCOPE_SYSTEM.

Multiple-processor scheduling



A common ready queue

All threads wait in one queue and any core takes the next. The queue is shared data, so access to it must be protected.



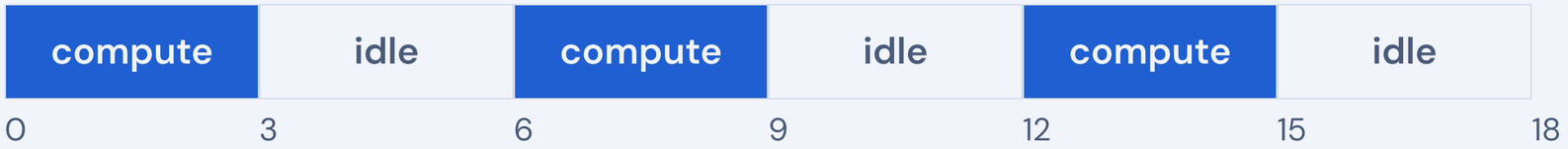
A queue for each core

Each processor schedules from its own private queue. This is the usual approach in symmetric multiprocessing.

Symmetric multiprocessing: each processor schedules itself. It is the standard on Windows, Linux, macOS, Android and iOS.

Multicore processors and hardware threads

ONE THREAD ON ONE CORE (IDLE = MEMORY STALL)



Memory stall

A processor is much faster than memory. A thread can spend half its time waiting for data to arrive.

Hardware threads

A core is given two or more hardware threads. When one stalls, the core switches to another. Intel calls this hyper-threading.

Two levels of scheduling

The operating system chooses a software thread for each hardware thread. The core chooses which hardware thread to run.

Load balancing and processor affinity



Load balancing

Keep the work evenly spread. Push migration: a task checks the load now and then, and moves threads away from a busy core. Pull migration: an idle core pulls a waiting thread from a busy one.



Processor affinity

A thread that ran on a core has its data in that core's cache. Moving it wastes that. Soft affinity: the system tries to keep it there. Hard affinity: the process names the cores it may use.

The two pull in opposite directions. On a NUMA system, a thread should also stay near the memory it uses.

PART 4

Real-time CPU scheduling

When a late answer is a wrong answer.

Soft and hard real-time systems

PREFERENCE

Soft real-time

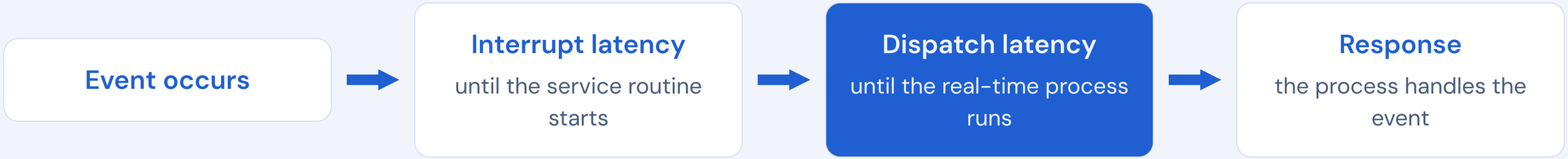
Critical processes are given preference over others, but there is no guarantee about when they will be scheduled.

GUARANTEE

Hard real-time

A task must be serviced by its deadline. Service after the deadline is the same as no service at all.

Latency



Interrupt latency

The time from the arrival of an interrupt to the start of the routine that services it.

Dispatch latency

The time to take the current process off the CPU and switch to the real-time one. It includes preempting any process in the kernel and releasing resources the real-time process needs.

Periodic tasks

A periodic task needs the CPU at *constant intervals*.

t

Processing time

How long it needs the CPU each time.

d

Deadline

When it must be finished.

p

Period

How often it arrives. Its rate is 1 / p.

Always $0 \leq t \leq d \leq p$. A real-time scheduler must be priority-based and preemptive.

Rate-monotonic scheduling

Task	Period	Time
P1	40	15
P2	80	30

ONE CYCLE OF 80



CPU utilization = 15/40 + 30/80 = 75%. Both tasks meet every deadline.

Static priorities: the shorter the period, the higher the priority. P1 preempts P2 at time 40. With N tasks, deadlines are guaranteed only up to a utilization of N(2^(1/N) - 1): about 83% for two tasks.

Earliest-deadline-first

Task	Period	Time
P1	40	20
P2	70	32

Utilization = 20/40 + 32/70 = 96%.

Rate-monotonic fails here: P2 has run for only 30 of its 32 units when its deadline arrives at time 70.

EDF assigns priorities dynamically. The earlier the deadline, the higher the priority. At time 40, P2's deadline (70) is earlier than P1's next one (80), so P2 is not preempted and finishes at 52.

In theory EDF can schedule any set of tasks whose total utilization is at most 100%.

Proportional share and POSIX real-time



Proportional share

T shares are divided among all applications. An application with N shares is given N / T of the processor time. A new application is admitted only if enough shares remain.



POSIX real-time classes

SCHED_FIFO: first-come, first-served, with no time slicing among threads of equal priority. SCHED_RR: the same, with time slicing among threads of equal priority.

PART 5

Scheduling in real operating systems

Linux, Windows and Solaris.

Linux: the Completely Fair Scheduler

Scheduling classes

Each class has a priority, and the scheduler picks the highest-priority task of the highest class. Two classes are standard: default (CFS) and real-time.

A proportion, not a quantum

CFS gives each task a proportion of CPU time based on its nice value, from -20 to +19. A lower nice value means a larger share.

Virtual run time

Each task has a vruntime: how long it has run, weighted by its priority. CFS always runs the task with the smallest vruntime.

A balanced tree

Runnable tasks are kept in a red-black tree ordered by vruntime. The next task to run is the leftmost node.

Windows scheduling

32

Priority levels

A queue for each level. The dispatcher runs the highest-priority ready thread, and preempts it if a higher one becomes ready.

1–15

Variable class

The priority can change. It is lowered when a thread uses its whole quantum, and raised when it returns from waiting.

16–31

Real-time class

Fixed priorities.

The window in the foreground gets a larger quantum, so the program the user is working with responds faster.

Solaris scheduling

Priority-based, with six classes. The default is time sharing.

A dispatch table gives, for each priority, the time quantum and the new priority after the quantum is used or after a sleep.

The higher the priority, the shorter the quantum. A thread that uses its quantum is moved down. A thread that wakes from waiting is moved up: good response for interactive work.

Class	Used for
Real time	Guaranteed response
System	Kernel threads
Time sharing	The default
Interactive	Windowing applications
Fair share	CPU shares
Fixed priority	Unchanging priorities

PART 6

Algorithm evaluation

How do we choose an algorithm for a particular system?

Four ways to evaluate

01

Deterministic modelling

Take one fixed workload and compute each algorithm's performance on it, as we did above. Simple and exact, but true only for that workload.

02

Queueing models

Describe arrivals and bursts by probability distributions, and compute averages with mathematics.

03

Simulation

Program a model of the system and feed it random data or traces recorded from a real system.

04

Implementation

Build it and measure it in real use. The only fully accurate way, and the most costly.

Little's formula

$$n = \lambda \times W$$

n is the average queue length. λ is the average arrival rate. W is the average waiting time in the queue.

In a steady state, the number of processes leaving the queue equals the number arriving. The formula holds for any scheduling algorithm and any arrival distribution.

TRY IT

Example

On average 5 processes arrive every second, and each waits 3 seconds in the queue. Then the queue holds 5 × 3 = 15 processes on average.

Key takeaways

01

Five criteria

Utilization, throughput, turnaround, waiting time and response time.

02

No best algorithm

SJF minimises average waiting. Round robin gives good response. Priorities need aging.

03

Real systems combine them

Multilevel feedback queues, a queue for each core, load balancing and affinity.

04

Real time needs guarantees

Rate-monotonic uses static priorities. Earliest-deadline-first uses dynamic ones.

Next session

COMING UP

Critical section and synchronization

Race conditions, the critical-section problem, Peterson's solution, hardware support and mutex locks.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 5.